TOOLKIT · BACKEND ENGINEERING HANDBOOK

GRAFANA COMPLETE HANDBOOK

Dashboards, Visualization, Prometheus Data Sources, Alerting & Production Observability

Phase	III — Production Observability
Tool ID	11 of 13
Curriculum Scope	Complete Single-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/grafana/ (01 markdown source)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, SREs, Systems Engineers
Engineering Focus	Prometheus Data Sources, Four Golden Signals, Panel Design, Dashboard Variables, Transformations & Observability Triage

This comprehensive handbook provides an authoritative, production-grade guide to **Grafana** for backend and site reliability engineering. Spanning three rigorous pedagogical layers—Mental Models, Practical Engineering, and Production Incident Mastery—it equips engineers to design intuitive operational dashboards, integrate Prometheus data sources securely, visualize the Four Golden Signals (Latency, Traffic, Errors, Saturation), select appropriate panel types (Time Series, Stat, Gauge, Table), master dashboard variables for dynamic multi-service scoping, apply post-query transformations, configure Grafana-managed vs Prometheus-managed alerting, provision dashboards as code, and systematically diagnose 'No data' anomalies, slow panel queries, and misleading visualizations.

Curriculum Structure & Core Topics

Part I — Dashboards, Visualization, Alerting & Production Observability: Visualization layer mental models, Prometheus integration, Four Golden Signals, panel architecture, PromQL dashboard queries (rate, 5xx ratios, p95 latency, heap ratios), variables and dynamic templating, transformations, annotations, Grafana-managed alerting vs Alertmanager, dashboard provisioning as code, and production troubleshooting runbooks.

PART I: DASHBOARDS, VISUALIZATION, ALERTING & PRODUCTION OBSERVABILITY

Complete Module · Backend Engineering Handbook

Grafana — Dashboards, Visualization, Alerting & Production Observability

Backend Engineering Tools & Infrastructure Handbook

Source file: `content/grafana/01-grafana-dashboards-visualization-alerting-production.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain Grafana's role in a production observability stack.
- Connect Grafana to Prometheus and build useful backend dashboards.
- Design panels for request traffic, error rates, latency, and infrastructure health.
- Use variables, dashboard transformations, and time ranges effectively.
- Understand Grafana alerting and its relationship with Prometheus alerting.
- Troubleshoot empty dashboards, incorrect queries, and misleading visualizations.
- Build a practical monitoring dashboard for a Spring Boot REST API.
- Answer common SDE-1 and backend engineering interview questions.

This single chapter covers all three required layers:

1. **Fundamentals & Mental Models**
2. **Practical Engineering & Workflows**
3. **Production Engineering & Interview Mastery**

Layer 1 — Fundamentals & Mental Models

1. What Is Grafana?

The visualization and operational analysis layer

Grafana is an observability and visualization platform that lets engineers query data sources, build dashboards, explore metrics, and configure alerting workflows.

Think of the backend monitoring stack as a team:

Layer / Component	Role in the Observability Pipeline
Spring Boot + Micrometer	Produces application metrics such as request counts, latency, and JVM memory.
Prometheus	Scrapes, stores, and queries time-series metrics.
Grafana	Turns queries into dashboards that help engineers understand system behavior.

Mental model: Prometheus is commonly the metrics collection and storage layer; Grafana is the visualization and exploration layer.

Grafana does not need to store the metrics itself when it queries Prometheus. It can connect to multiple supported data sources, including systems for metrics, logs, and traces.

2. Why Dashboards Matter

Imagine a user reports that your Orders API is slow.

Without a dashboard, you might manually inspect application logs, CPU usage, database metrics, and request-duration measurements separately.

A well-designed dashboard helps you answer four questions quickly:

1. **Is the service healthy?**
2. **What changed?**
3. **Where is the bottleneck?**
4. **Is the problem affecting users?**

A useful dashboard is not a wall of colorful charts. It is an organized interface for making operational decisions.

3. The Four Golden Signals

A practical starting point for a backend service is the four golden signals:

Golden Signal	Concrete Meaning	Backend Production Example
Latency	How long requests take to complete, including tail latency distributions.	p95 or p99 response time (`histogram_quantile`)
Traffic	How much demand is placed on the service.	Requests per second (`rate(http_requests_total[5m])`)
Errors	The rate of requests that fail explicitly or implicitly.	HTTP 5xx server-error ratio
Saturation	How close a constrained resource is to its useful capacity limit.	CPU throttling, database connection pool exhaustion, JVM heap limits

These signals are complementary. Low CPU utilization does not prove that an API is healthy; the database connection pool might be exhausted while the application waits.

4. Dashboards, Panels, Queries, and Data Sources

Concept	Meaning
Data source	The system Grafana queries, such as Prometheus
Dashboard	A collection of related visualizations
Panel	An individual visualization or query result
Query	An expression that retrieves or calculates data
Variable	A reusable dashboard parameter, such as a service name
Transformation	A post-query operation that changes how returned data is presented
Annotation	An event marker, such as a deployment or incident
Alert rule	A condition evaluated to determine whether action may be required

A panel generally follows this path:

```
Data source → Query → Returned data → Optional transformation → Visualization
```

If the query is wrong, a beautiful chart will still be wrong.

5. Choose the Right Visualization

Avoid choosing a visualization merely because it looks impressive. Select it based on the question the engineer needs to answer:

Time Series

Use for continuous numerical changes over time: request rates, latency over time, CPU utilization, resident memory, and queue depth.

Stat

Use for a concise current value or key performance indicator: current request rate, current error ratio, or global uptime percentage.

Gauge

Use for a value compared against a meaningful range or threshold: disk utilization, thread-pool utilization, or memory limits.

Table

Use for multi-dimensional comparisons: ranked slow endpoints, instance health breakdowns, or error count lists.

Layer 2 — Practical Engineering & Workflows

6. Run Grafana Locally

If you already have Prometheus running from the previous handbook chapters, add Grafana to your existing Docker Compose project.

```
services:
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana

volumes:
  grafana-data:
```

This is a minimal development example. In a production deployment, pin a tested image version, configure authentication and access controls, and manage credentials securely.

Start the service:

```
docker compose up -d grafana
docker compose ps
docker compose logs -f grafana
```

Open `http://localhost:3000` in your browser and sign in using the credentials configured for your instance.

Do not assume the default development credentials are appropriate for a deployed environment.

7. Connect Grafana to Prometheus

In the Grafana interface:

1. Open **Connections** or **Data sources**.
2. Add a Prometheus data source.
3. Enter the Prometheus URL.
4. Save and test the connection.

If Grafana and Prometheus are running in the same Compose network, the URL may be:

```
http://prometheus:9090
```

Use the actual Compose service name.

Important: `localhost` inside a Grafana container refers to that container itself, not the Prometheus container and not necessarily your host machine.

This is one of the most common container-networking mistakes.

8. Build Your First Backend Dashboard

Create a dashboard called `Spring Boot API — Operations`.

Start with these panels:

Panel	What it answers
Request rate	How much traffic are we receiving?
5xx error ratio	Are server errors increasing?
p95 latency	Are slow requests affecting users?
JVM heap usage	Is the application approaching memory pressure?
CPU usage	Is processing capacity constrained?
Target availability	Is Prometheus successfully scraping the service?

A good dashboard should have a logical reading order:

1. Overall service health.
2. Traffic, errors, and latency.
3. Runtime and dependency health.
4. Detailed breakdowns for investigation.

9. Essential PromQL Queries for Grafana

These examples assume the corresponding metrics and labels exist in your application. Spring Boot and Micrometer metric names can vary with configuration and versions, so verify the actual series first.

A. Request rate

```
sum(
  rate(http_server_requests_seconds_count{
    job="backend-api"
  }[5m])
)
```

This estimates the average request rate over the previous five minutes.

B. Server-error ratio

```
sum(rate(http_server_requests_seconds_count{
  job="backend-api",
  status=~"5.."
}[5m]))
/
clamp_min(
  sum(rate(http_server_requests_seconds_count{
    job="backend-api"
  }[5m])),
  0.001
)
```

Format the result as a percentage in Grafana.

At low or zero traffic, a ratio alone can be misleading. Consider displaying request volume alongside it and defining an appropriate minimum-traffic condition for alerts.

C. p95 latency

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(http_server_requests_seconds_bucket{
      job="backend-api"
    }[5m])
  )
)
```

This calculates an estimated 95th-percentile latency from histogram buckets. It requires a compatible histogram metric with the `le` bucket label.

D. Target availability

```
up{job="backend-api"}
```

A value of `1` means the most recent scrape succeeded; `0` means it failed.

An absent series is a different situation: the target may not be discovered or configured at all.

E. JVM heap usage ratio

If your application exports the expected JVM memory metrics:

```
sum(jvm_memory_used_bytes{
  job="backend-api",
  area="heap"
})
/
clamp_min(
  sum(jvm_memory_max_bytes{
    job="backend-api",
    area="heap"
  }),
  1
)
```

Treat this as an illustrative aggregation. Verify that the available maximum-memory metric and labels have the semantics you expect. Heap behavior should also be interpreted alongside garbage collection, allocation, and process-level memory.

10. Configure Panel Settings Correctly

For each panel, configure:

- **Title:** State what the measurement represents.
- **Unit:** Seconds, bytes, requests per second, or percent.
- **Legend:** Distinguish services or instances where useful.
- **Time range:** Use a range suitable for the question.
- **Thresholds:** Reflect a meaningful operational boundary.
- **Description:** Explain the metric and its limitations.

- **Query behavior:** Avoid unnecessary high-cardinality breakdowns.

For a latency panel, a unit of seconds or milliseconds is essential. A number without units invites misinterpretation.

For an error ratio, a result of `0.025` means 2.5%, not 0.025%, when the expression returns a fraction.

11. Variables: One Dashboard, Multiple Services

Variables let users change the service or environment without duplicating a dashboard.

For example, create a variable named `job` whose values come from the Prometheus `job` label.

You can then write:

```
sum(rate(http_server_requests_seconds_count{
  job="$job"
}[5m]))
```

A variable makes the dashboard reusable across services.

For multi-value variables, use a regex matcher when appropriate:

```
sum(rate(http_server_requests_seconds_count{
  job=~"$job"
}[5m]))
```

The exact variable syntax and escaping behavior depend on Grafana's configuration. Test single-value, multi-value, and all-value selections.

Avoid creating variables from unbounded labels such as request IDs or user IDs. Doing so can make the dashboard slow and difficult to use.

12. Dashboard Transformations and Annotations

Transformations

Grafana transformations can reshape returned data for presentation.

Examples include:

- Joining results from separate queries.
- Renaming fields.
- Filtering rows.
- Sorting values.
- Reducing a series to a single value.

Transformations are useful for presentation, but they should not obscure the underlying calculation. When possible, keep important operational logic explicit in the query or in a documented recording rule.

Annotations

Annotations mark events on a dashboard, such as:

- A deployment.

- A configuration change.
- A database migration.
- A known incident.

They help correlate a change in metrics with a change in system behavior.

For example, a latency spike that begins immediately after a deployment is useful evidence. It is not, by itself, proof that the deployment caused the spike.

13. Grafana Alerting

Grafana can support alert rules that evaluate data-source queries and send notifications through configured contact points and notification policies.

A typical flow is:

```
Metric data → Alert query → Condition evaluation → Notification policy → Contact point
```

A contact point represents a notification destination, such as an email integration or webhook.

Grafana alerting versus Prometheus alerting

Both systems can be used to alert on metrics. Choose an approach deliberately rather than creating duplicate rules everywhere.

Consideration	Prometheus rules + Alertmanager	Grafana-managed alerting
Rule evaluation	Prometheus	Grafana-managed alerting evaluates configured rules
Data focus	Prometheus time series	Supported Grafana data sources, depending on configuration
Notification management	Alertmanager routing and grouping	Grafana notification policies and contact points
Best fit	Metrics-focused, configuration-as-code workflows	Teams that manage alerting centrally through Grafana
Main caution	Ensure Alertmanager routing and delivery work	Ensure evaluation, data-source access, and notification routing are reliable

The systems are not mutually exclusive in every architecture. Avoid unintentionally alerting twice on the same condition.

Layer 3 — Production Engineering & Interview Mastery

14. Production Dashboard Design

A production dashboard should help an engineer make a decision quickly.

Use these principles:

Lead with user impact. Put availability, error ratio, latency, and traffic near the top.

Keep panels purposeful. Every panel should answer a question or help investigate a known failure mode.

Use consistent units. Do not mix milliseconds and seconds without clear labels.

Preserve context. Include environment, service, and instance filters when useful.

Avoid misleading averages. A low average latency can hide a small number of extremely slow requests.

Use meaningful thresholds. A threshold should connect to an SLO, capacity limit, or operational response—not an arbitrary color choice.

Make drill-down possible. Link overview panels to more detailed dashboards or relevant logs and traces when your stack supports it.

Treat dashboards as code where practical. Version-controlled provisioning makes changes reviewable, repeatable, and easier to recover.

15. Dashboard Provisioning and Version Control

For repeatable environments, configure Grafana resources through provisioning or another supported configuration-as-code workflow.

Benefits include:

- Reproducible dashboards.
- Reviewable changes.
- Easier recovery after instance replacement.
- Consistency between development, staging, and production.
- Reduced manual setup.

A basic repository layout might look like:

```
monitoring/
■■■ prometheus/
■   ■■■ prometheus.yml
■   ■■■ alert-rules.yml
■■■ grafana/
    ■■■ provisioning/
    ■   ■■■ datasources/
    ■   ■■■ dashboards/
    ■■■ dashboards/
        ■■■ backend-api.json
```

This is a suggested layout, not a mandatory Grafana directory structure. The actual provisioning paths depend on the image, configuration, and deployment approach.

Avoid storing passwords, API tokens, or private webhook URLs in a public repository. Treat dashboard changes as operational changes: review, validate, deploy, and verify.

16. Troubleshooting Scenarios

Scenario A — Grafana says “No data”

Check in this order:

1. Is the Prometheus data source configured correctly?
2. Can Grafana reach Prometheus over the network?
3. Does the metric exist in Prometheus?
4. Does the query use the correct metric name?
5. Do the label filters match actual series?
6. Is the selected dashboard time range appropriate?
7. Is the query returning an empty vector or `NaN`?
8. Are dashboard variables filtering everything out?

A successful data-source connection does not guarantee that every query returns data.

Scenario B — Dashboard is slow

Investigate:

- Expensive PromQL expressions.
- Excessive time-series cardinality.
- Too many panels refreshing simultaneously.
- Very long time ranges.
- Large table results.
- Repeated calculations that could use recording rules.
- Excessive dashboard refresh frequency.

Fix the query and data model before simply increasing server resources.

Scenario C — Dashboard shows 0% errors while users see failures

Possible causes include:

- Incorrect status-code labels.
- Missing instrumentation.
- A query that excludes the affected endpoint.
- A dashboard variable selecting the wrong service.
- Errors happening in a dependency or gateway not covered by the panel.
- A metric that counts only completed requests or does not include the failure type.

Compare the panel with the underlying metrics, application logs, gateway measurements, and actual request outcomes.

Scenario D — Grafana alert fires repeatedly

Investigate:

1. Whether the rule evaluates the intended series.
2. Whether the threshold is meaningful.
3. Whether transient conditions should have a pending period.
4. Whether the rule creates multiple alert instances.
5. Whether grouping and notification policies are appropriate.
6. Whether the condition should be expressed as an SLO or burn-rate alert instead.

Do not solve every repeated alert by increasing the threshold. You may be hiding a real reliability problem.

17. Security and Operational Reliability

Production Grafana should be treated as an operational interface with access to potentially sensitive data.

Follow these practices:

- Restrict access to the Grafana UI and APIs.
- Use appropriate authentication and role-based permissions.
- Protect credentials and contact-point secrets.
- Limit data-source permissions and network access.
- Avoid embedding secrets in dashboard URLs or exported JSON.
- Review dashboard changes before deployment.
- Back up or version-control important dashboards and provisioning configuration.
- Monitor Grafana availability and data-source failures.
- Test alert delivery and recovery behavior.
- Keep deployment versions and upgrades under change control.

A dashboard can reveal internal service names, deployment details, and operational patterns. Access control is therefore a production concern, not merely a UI preference.

18. Twenty Interview Questions and Answers

1. What is Grafana?

Grafana is an observability and visualization platform used to query data sources and build dashboards, explore data, and manage alerting workflows.

2. What is the difference between Grafana and Prometheus?

Prometheus commonly collects and stores metrics and provides PromQL. Grafana queries Prometheus and visualizes the results.

3. What is a Grafana data source?

A configured connection to a system from which Grafana retrieves data, such as Prometheus.

4. What is a dashboard panel?

An individual visualization or query result within a dashboard.

5. What are the four golden signals?

Latency, traffic, errors, and saturation.

6. Why is p95 latency often more useful than average latency?

It provides information about the slower portion of requests that an average may hide. It should still be interpreted alongside traffic volume and other latency measurements.

7. What is a Grafana variable?

A reusable dashboard parameter that lets users change a filter, such as service, environment, or job.

8. What is a dashboard transformation?

An operation that reshapes or modifies query results for presentation inside Grafana.

9. Why use annotations?

Annotations mark events such as deployments and incidents, helping engineers correlate events with metric changes.

10. What causes a Grafana “No data” panel?

Common causes include incorrect queries, label mismatches, unavailable metrics, time-range problems, variable filters, and data-source connectivity issues.

11. Can Grafana work without Prometheus?

Yes. Grafana supports multiple data sources; Prometheus is a common metrics source, not its only possible source.

12. Does Grafana need to store Prometheus metrics itself?

No. It can query the metrics stored by Prometheus.

13. What is the difference between Grafana alerting and Prometheus alerting?

Prometheus evaluates Prometheus rule expressions and commonly sends alerts to Alertmanager. Grafana-managed alerting can evaluate configured queries and manage notification policies through Grafana.

14. Why can a dashboard be misleading even when every query succeeds?

The queries may omit relevant traffic, aggregate incorrectly, use unsuitable units, or hide tail latency and failures.

15. How do you make dashboards reusable?

Use variables, consistent metric naming, reusable panels, and documented service filters.

16. Why is dashboard provisioning useful?

It makes dashboards reproducible and allows changes to be reviewed, versioned, and deployed consistently.

17. What causes high dashboard query latency?

Expensive queries, high cardinality, excessive panel refreshes, large result sets, and long time ranges are common causes.

18. Why should a dashboard include traffic alongside error ratio?

A ratio based on very few requests can be misleading. Traffic provides context for interpreting the error measurement.

19. How would you investigate an API latency incident using Grafana?

Start with latency, traffic, and errors; compare the time window against deployment annotations; then drill down into application, runtime, database, cache, and downstream-service metrics.

20. What makes a dashboard production-ready?

It provides trustworthy measurements, useful units and thresholds, clear service context, appropriate access control, reproducible configuration, and a fast path from symptom to investigation.

19. Hands-On Capstone

Build an operational dashboard for a Spring Boot Orders API.

Required panels

- [ ] Request rate over time.
- [ ] HTTP 5xx ratio.
- [ ] p95 latency.
- [ ] JVM heap usage.
- [ ] CPU or process-resource health.
- [ ] Prometheus target availability.
- [ ] A service or environment variable.
- [ ] A deployment annotation or documented event marker.

Required operational exercises

- [ ] Connect Grafana to Prometheus.
- [ ] Verify the data source.
- [ ] Test each query independently in Prometheus.
- [ ] Configure correct units and labels.
- [ ] Diagnose an intentionally incorrect label filter.
- [ ] Investigate an empty panel.
- [ ] Compare average latency with tail latency.
- [ ] Explain when a recording rule would improve a repeated query.
- [ ] Choose a consistent alerting approach and explain why.
- [ ] Document the dashboard's limitations and next diagnostic steps.

Definition of done

You can open the dashboard during an incident, establish whether users are affected, identify the most relevant signals, and explain how you would investigate the underlying cause.

Final Mastery Checklist

- [ ] Explain Grafana's role in the observability stack.
- [ ] Connect Grafana to a Prometheus data source.
- [ ] Build panels using appropriate PromQL queries.
- [ ] Select meaningful visualizations and units.
- [ ] Use dashboard variables effectively.
- [ ] Understand transformations and annotations.
- [ ] Explain Grafana-managed versus Prometheus-managed alerting.
- [ ] Diagnose empty, incorrect, or slow dashboards.
- [ ] Apply production access controls and configuration management.
- [ ] Build and defend the Orders API dashboard.

Grafana Handbook Completion Protocol

This is the **single planned Grafana content chunk**. The curriculum for Grafana is now complete in authored content.

Repository and release work remain separate and must be verified:

1. Save this chapter to `content/grafana/01-grafana-dashboards-visualization-alerting-production.md`.
2. Review the examples and ensure they match the actual metrics exposed by the application.
3. Verify the source file is complete.
4. Generate the **single complete Grafana PDF** at `docs/grafana/BEH_GRAFANA_COMPLETE_HANDBOOK.pdf`.
5. Open and verify the final PDF.
6. Update the README using verified progress only.
7. Once the source and final PDF are saved and verified, commit with the exact message:

```
Grafana done
```

Do not generate a PDF per chunk, and do not claim repository changes, PDF generation, or a Git commit until those actions have actually been performed and verified.